Cupcake

Dokumentation af webshop for Copenhagen Cupcakes

2. semester · Datamatiker · Gruppe E

Projektperiode: 2.–12. november 2026

Rapportversion: 9. november 2026

Deltager	GitHub	Studie-e-mail
Lucas Beltner	Borntofart	[lube1001@stud.ek.dk]
Sofus Juul	Langemanden	[cph-sj676@stud.ek.dk]
Philip Tranto	Nerothras	[phtr1000@stud.ek.dk]

Klasse: Datamati (26as20177Vn3) - GitHub-link: [Tryk her] - Videodemonstration: [klick her]

1. Indledning 1

1.1 Teknologivalg 2
2. Krav 2
2.1 Funktionelle krav og acceptkriterier 2
2.2 Ikke-funktionelle krav 3
3. Domænemodel og ER-diagram 3
3.1 Domænemodel 3
3.2 ER-diagram 3
4. Mockups og designproces 4
5. Navigationsdiagram 5
6. Implementation 6
6.1 MVC og SRP – Strukturering af koden 6
6.2 Session og kurv 6
6.3 Login og adgangskontrol 6
6.4 Betaling og databasetransaktion 7
6.5 Validering og fejl 7
7 Test 7
7.1 Unit tests 7
7.2 Black-box-test 8
7.3 Brugertest 8
7.4 Vurdering af test 8
8. Status på implementering 9
9. Proces 9
9.1 Samarbejde og planlægning 9
9.2 Roller og ansvar 10
9.3 Git-strategi, kvalitet og refleksion 10
10. Konklusion 11
Bilag A – ER-diagram 12
Bilag B – Mockup og designiteration 13
Bilag C – Mockup og designiteration 14
Bilag D – Mockup og designiteration 15
Bilag E – Mockup og designiteration 16
Bilag F – Teknisk dokumentation og afleveringskontrol 17

1. Indledning

Dette projekt har til formål af at udvikle webshop til Copenhagen Cupcakes, som ønsker at gøre det nemmere for deres kunder at bestille cupcakes online. Websitet skal kunne give kunderne mulighed for selv at sammensætte deres cupcakes ved at vælge forskellige bunde og toppings, tilføje dem til en indkøbskurv og derefter kunne gennemføre en bestilling. På den måde kan kunderne bestille deres cupcakes hjemmefra og efterfølgende afhente dem i butikken.

Webshoppen har også til formål at gøre det lettere for virksomheden at administrere deres bestillinger og kunder. Systemet indeholder derfor en personlig profil, hvor brugerne kan oprette en konto, logge ind og foretage bestillinger, samt en administrationsdel, hvor medarbejdere med administratorrettigheder kan få overblik over kunder og ordrer.

Projektet er udviklet med Java, Javalin, Thymeleaf, PostgreSQL, HTML og CSS. Projektet s kode er arbejdet ud fra MVC-arkitekturen (Model-View-Controller) og principperne fra Single Responsibility Principle (SRP). Årsagen til dette er at denne struktur hjælper med at adskille systemets forskellige ansvarsområder og skaber en tydelig struktur gennem hele projektets kode.

Dertil er der arbejdet med brugergrænsefladens design gennem flere iterationer af mockups, hvor der er arbejdet ud fra principperne fra Laws of UX og gestaltlovene. Målet har været at udvikle en webshop, som både er funktionel og intuitiv at anvende.

1.1 Teknologivalg

Applikationen er udviklet i Java 21 og bruger Javalin 6.3.0 til HTTP-ruter og Thymeleaf 3.1.2 til at generere HTML-sider. Data lagres i PostgreSQL via JDBC-driveren 42.7.4. HikariCP 5.1.0 står for forbindelsespuljen, og jBCrypt 0.4 bruges til håndtering af adgangskoder. Brugergrænsefladen er lavet med HTML og CSS. Maven styrer afhængighederne, og GitHub er anvendt til versionsstyring. De nævnte versionsnumre kan dokumenteres i pom.xml; versionsnumre for IntelliJ, PostgreSQL-server og pgAdmin skal tilføjes, hvis de skal oplyses præcist.

Arkitekturen er opdelt i controllere, services, persistence/mappere, entities og templates. Dette gøres da det giver et naturligt skel mellem håndtering af HTTP-requests, forretningsregler, SQL og præsentation. Opdelingen har gjort det lettere at arbejde med de enkelte user stories, samtidig med at holde styr på, hvor en ændring hører til.

2. Krav

Kundens krav og vision er, at en bestilling skal kunne gennemføres uden at kunden behøver stå i kø i butikken. Derfor er de centrale funktioner registrering, login, sammensætning af varer, indkøbskurv, betaling og ordrehistorik. For butikken er administration af kunder og ordrer tilsvarende vigtig.

2.1 Funktionelle krav og acceptkriterier

US-1 – Bestille og betale cupcakes med valgfri bund og top. Acceptkriterier anvendt til vurdering: Kunden kan vælge bund, topping og antal; prisen beregnes; betaling med tilstrækkelig saldo gemmer ordren.

US-2 – Oprette en konto/profil. Acceptkriterier anvendt til vurdering: En kunde kan oprette sig med e-mail og adgangskode; ugyldige eller eksisterende oplysninger afvises.

US-3 – Se ordrelinjer i en indkøbskurv. Acceptkriterier anvendt til vurdering: Kurven viser de valgte varer, antal og samlet pris.

US-4 – Logge ind som kunde eller administrator. Acceptkriterier anvendt til vurdering: Login med e-mail og adgangskode virker, og den aktuelle bruger fremgår af siden.

US-5 – Administrator kan se alle ordrer. Acceptkriterier anvendt til vurdering: En administrator kan åbne en oversigt over ordrer.

US-6 – Administrator kan se kunder og deres ordrer. Acceptkriterier anvendt til vurdering: Administratoren kan tilgå en kundeoversigt med ordreoplysninger.

US-7 – Fjerne en ordrelinje fra kurven. Acceptkriterier anvendt til vurdering: En kunde kan fjerne en valgt linje, og totalen opdateres.

US-8 – Administrator kan fjerne en ordre. Acceptkriterier anvendt til vurdering: En administrator kan slette en ordre, som ikke længere skal være i systemet.

2.2 Ikke-funktionelle krav

Data skal gemmes i en relationel database, som som udgangspunkt følger tredje normalform. Kundesaldo tilføjes direkte i PostgreSQL via pgAdmin, og systemet skal bruge Java, Javalin, Thymeleaf, PostgreSQL, HTML og CSS. Koden skal deles på GitHub. Designet bør kunne anvendes på både laptop og mobil, men opgaven accepterer en laptopløsning, hvis responsivitet bliver for tidskrævende. Vi har lagt vægt på en ensartet navigation og tydelige primære handlinger.

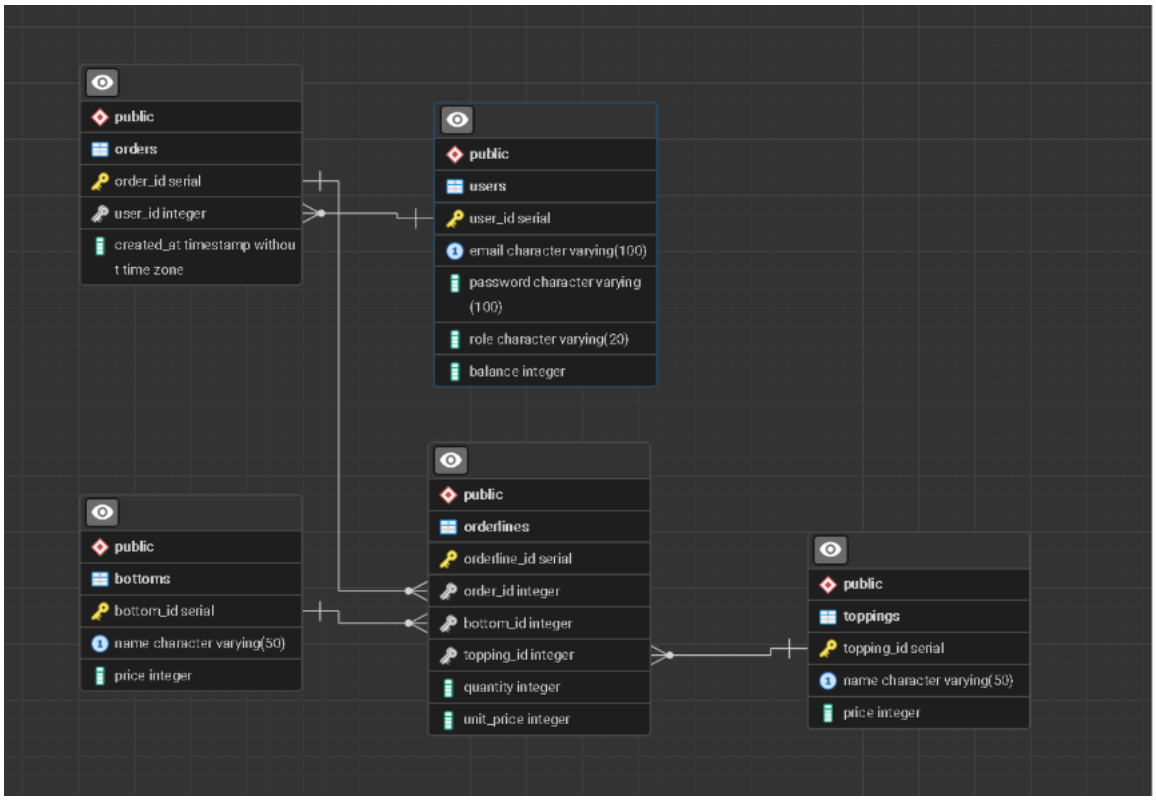
3. Domænemodel og ER-diagram

3.1 Domænemodel

I domænet er de vigtigste begreber Kunde, Ordre, Ordrelinje, Bund og Topping. En kunde nemlig muligheden for at have flere ordre ad gangen, mens en ordre tilhører én kunde. En ordre indeholder en eller flere ordrelinjer, og hver linje beskriver et antal cupcakes med én bestemt bund og én topping. En bund eller topping kan anvendes på mange forskellige ordrelinjer. Administrator er i vores løsning en Admin rolle som brugerrolle frem for være et domæneobjekt.

Den model afspejler, at en ordre er en samlet bestilling, mens ordrelinjen er den enkelte varekombination. Opdelingen er der på grund af en kunde kan købe forskellige typer cupcakes i samme ordre.

3.2 ER-diagram



Figur 1. Gruppens ER-diagram.

Databasen består af users, orders, orderlines, bottoms og toppings. users har en primærnøgle user_id og indeholder en e-mail, adgangskodehash, rolle og saldo. orders har en fremmednøgle til users, mens orderlines har fremmednøgler til både orders, bottoms og toppings. De enkelte bund- og toppingtyper er sat i separate tabeller, så der undgås at komme gentagelser i ordrelinjerne som ved navn og pris.

I SQL-scriptet anvendes "serial" som primærnøgle i alle fem tabeller. E-mail er "Unique", og bund- og toppingnavne er også sat som unikke. Quantity, altsp mængden, har "Check", hvilket hindrer ordrelinjer med nul eller negativt antal. Fremmednøglerne fra orders til users, og fra orderlines til orders har ON DELETE CASCADE, hvilket betyder, at tilhørende ordrelinjer fjernes, så hvis en ordre bliver slette, fjernes det.

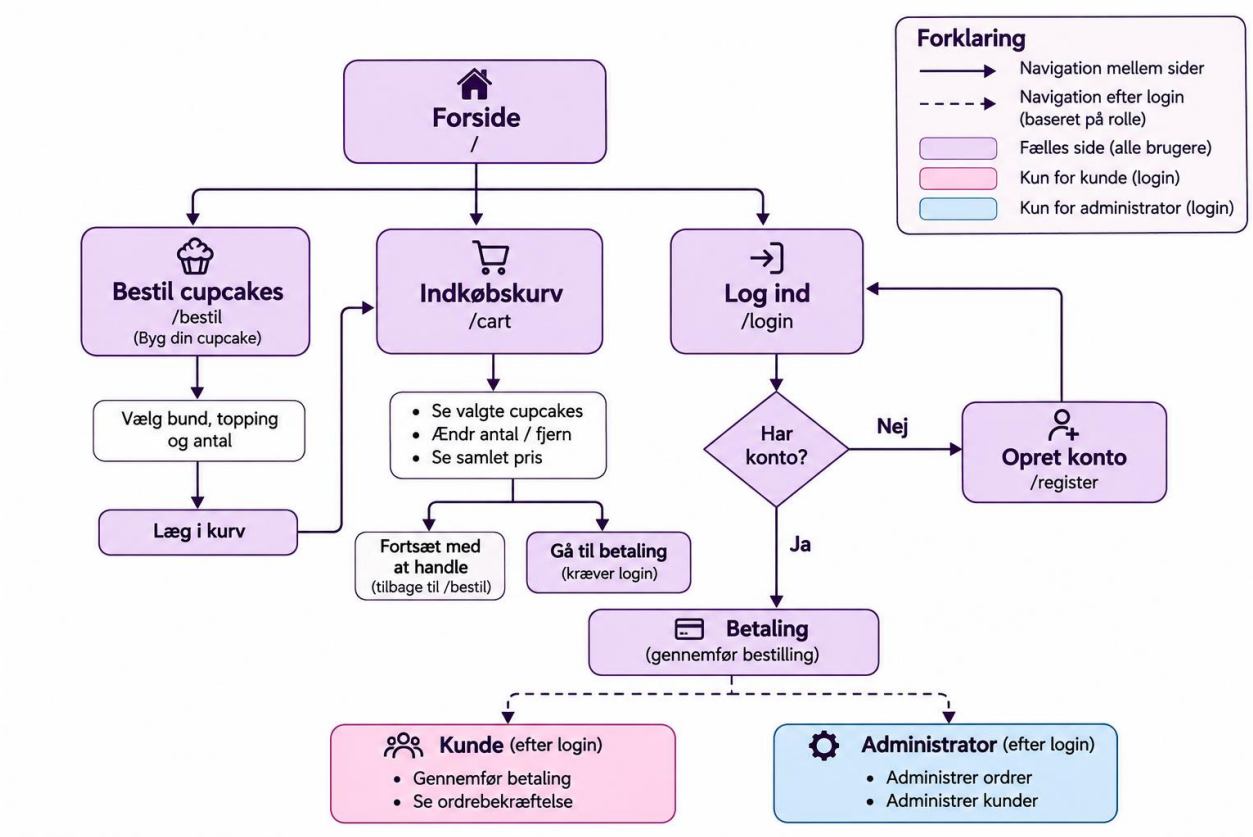
unit_price gemmes på ordrelinjen, selv om bottoms og toppings også indeholder priser. Eksempelvis hvis katalogprisen ændres senere, vil en tidligere betalt ordre ordre stadig vise den oprindelige stykpris. Dette gør at der undgås at genberegne gamle ordrer ud fra nye priser.

4. Mockups og designproces

Vi begyndte med håndtegnede skitser og en enkel papirprototype. For vores gruppe handlede arbejdet mest om, hvilke sider brugeren skulle møde samt hvor indholdet skulle placeres, f.eks. hvor ting som knapper og boxe, billeder osv. Skulle være. Derefter lavede vi digitale mockups i Figma og arbejdede os frem mod et mere gennemført udtryk. Gennem processen lavede vi flere versioner som gav os muligheden for at flytte elementerne på hjemmesiden, før vi brugte tid på at implementere dem. Se blandet andet Bilag B, C, D som viser den mere endelige design valg vi gik efter.

Et gennemgående valg i vores gruppe blev at de vigtigste knapper er lilla, mens navigationen ligger samme sted på siderne. Dette gjorde vi for at skabe genkendelighed og gør det lettere at forstå, hvilke handlinger der er primære. Derfor arbejdede ud fra Gestaltlovene om nærhed og lighed, hvor felter og oplysninger hører sammen, og knapper med samme formål får en ensartet "look". Vi har også arbejdet ud fra Hick's Law ved at gøre produktvalgene overskuelige og ikke have for meget tekst eller andet som ville gøre det for kompliceret for brugerne at traffe et valg. Dertil er Jakob's Law også indarbejdet i design processen da navigationen skal være genkendelig som fra andre websites. Det er nemlig vigtigt, da man som bruger skal kunne nemt genkende websitets opsætningen og derfor minde dem om andre websites som der allerede har navigeret rundt i før.

5. Navigationsdiagram



Fra forsiden kan brugeren gå til "Bestil cupcakes", "Indkøbskurv" og "Log ind". På bestillingssiden kan kunden vælge bund, topping og antal, hvorefter produkterne kan tilføjes til indkøbskurven. Her kan kunden se den samlede pris, fjerne produkter eller fortsætte til betaling.

Vi tog valget at kunder kan sammensætte cupcakes uden at være logget ind for ikke at bryde deres bestilling. Login bliver derfor først nødvendigt ved betaling, og nye kunder har så muligheden for at oprette en konto.

Navigationen er desuden opdelt efter brugerroller. Almindelige kunder kan gennemføre bestillinger, mens en admin har adgang til at administrere kunder og ordrer. I diagrammet er fælles sider markeret med lilla, kundefunktioner med lyserød og administratorfunktioner med blå.

Denne opdeling har vi lavet for at gøre det så overskueligt som muligt på hjemmesiden og for sikrer samtidig, at brugerne kun har adgang til de funktioner, deres rolle tillader.

6. Implementation

6.1 MVC og SRP – Strukturering af koden

Vi har udviklet vores projekt med udgangspunkt i **MVC-modellen (Model-View-Controller)** for at skabe en overskuelig struktur og adskille de forskellige ansvarsområder i koden.

Vores **controllers** håndterer brugerens handlinger og HTTP-requests. F.eks. står UserController for login og registrering, mens ShopController håndterer bestillinger og indkøbskurven. Vores services indeholder forretningslogikken, mens mappers håndterer kommunikationen med databasen. Vores entities repræsenterer systemets data, og Thymeleaf bruges til at vise hjemmesidens indhold.

Vi har samtidig arbejdet meget ud fra Single Responsibility Principle (SRP), som betyder, at hver klasse bør have ét tydeligt ansvarsområde. F.eks. håndterer UserController brugerens handlinger, UserService brugerrelateret logik og `UserMapper` databaseforespørgsler. Dette er især gjort for at holde klasserne nemme og overskuelige samt test af koden bliver derfor også nemmere.

6.2 Session og kurv

Ved login gemmer UserController brugeren i sessionen under navnet currentUser. ViewModel samler fælles data til skabelonerne, så siderne kan vise den nuværende bruger. Kurven opbevares ligeledes i sessionen under cart. Det gøre at kunden kan bruge og navigere på de forskellige sider uden at miste deres varer som allerede ligger i kurven. Sessionen fungerer altså som en midlertidig tilstand, mens gennemførte ordrer lagres permanent i databasen.

ViewModel indeholder desuden en funktion til at hente opdaterede brugeroplysninger. Det er relevant, fordi saldoen ifølge opgaven tilføjes direkte i databasen. Hvis vi kun brugte saldoen fra den oprindelige login-session, ville kunden kunne se et forældet beløb.

6.3 Login og adgangskontrol

Brugerhåndteringen benytter jBCrypt, så adgangskoder ikke skal sammenlignes eller lagres som almindelig tekst. Dette betyder at koden bliver brudt op i bytes og derved bliver mere sikker og svært for en hacker at stjæle brugernes adgangkode. Administratorfunktionerne ligger i AdminController, hvor adgang kontrolleres via brugerens rolle. En almindelig kunde skal ikke kunne få adgang til administratorens ordrer blot ved at skrive en URL manuelt.

Login og registrering viser fejlbeskeder ved ugyldige oplysninger. En god fejlbesked skal være forståelig og samtidig undgå at afsløre unødvendige tekniske detaljer om databasen. Ved logout fjernes den aktive session, så den tidligere bruger ikke fortsat opfattes som logget ind.

6.4 Betaling og databasetransaktion

Et af de vigtigste kodevalg findes i OrderMapper.createOrder. Her bliver der lavet saldoopdatering, oprettelse af ordre og indsættelse af ordrelinjer i én databasetransaktion. Metoden slår autocommit fra og bruger commit, når alle trin lykkes. Ved en fejl udføres et såkaldt "rollback", som beskytter mod en situation, hvor kundens saldo bliver trukket, uden at ordren bliver gemt.

Saldoen reduceres med en SQL-opdatering, som kun lykkes, hvis balance >= den samlede pris. Det er mere robust end alene at kontrollere saldoen på websiden, fordi kontrollen sker direkte i databasen i forbindelse med betalingen. Ordrelinjerne indsættes samlet med en batch, og ordre-ID hentes fra databasen efter oprettelsen. Dermed kan hver ordrelinje knyttes til den rigtige ordre.

6.5 Validering og fejl

Vi har arbejdet med validering for at sikre, at systemet håndterer brugerinput korrekt. Eksempelvis kontrollerer vi, at e-mailadresser er unikke, og at antallet af cupcakes er gyldigt. Vi bruger også PreparedStatement til vores SQL-forespørgsler, hvilket hjælper med at beskytte mod SQL-injektion.

Til fejlhåndtering anvender vi vores egen DBException, som gør det nemmere at håndtere databasefejl. Ved betaling bruger vi databasetransaktioner, så hele handlingen bliver annulleret, hvis der opstår en fejl undervejs.

På den måde sikrer vi, at fejl bliver håndteret korrekt, og at databasen ikke ender med ufuldstændige ordrer.

7 Test

For at sikre at vores webshop fungerer som forventet, har vi arbejdet med forskellige former for test gennem udviklingsprocessen. Vi har både anvendt unit tests, black-box-test og brugertest. Formålet har været at kontrollere systemets funktionalitet, håndtering af fejl og brugervenlighed.

7.1 Unit tests

Vi har implementeret 13 unit tests ved hjælp af JUnit 5. Testene er fordelt på tre testklasser, som undersøger forskellige dele af systemets forretningslogik.

I OrderTotalTest har vi lavet fem tests, der kontrollerer beregningen af priser i indkøbskurven og på ordrer. Vi tester blandt andet, om prisen på en ordrelinje bliver beregnet korrekt ud fra bund, topping og antal. Derudover kontrollerer vi, om den samlede pris bliver opdateret korrekt, når produkter tilføjes eller fjernes fra kurven. Vi tester også, at en tom kurv har en samlet pris på 0 kr.

En vigtig test undersøger, om en tidligere oprettet ordre anvender den gemte stykpris frem for den aktuelle produktpris. Dette er relevant, fordi ændringer i produktpriser ikke skal påvirke allerede oprettede ordrer.

I OrderServiceTest har vi lavet tre tests, der kontrollerer valideringen af antallet af cupcakes. Systemet tillader mellem 1 og 50 cupcakes ad gangen. Vi tester derfor både gyldige værdier og ugyldige værdier som 0, 51 og negative tal. Derudover kontrollerer vi, at systemet kaster en IllegalArgumentException, hvis man forsøger at tilføje et ugyldigt antal til indkøbskurven.

I UserServiceTest har vi lavet fem tests, der undersøger loginfunktionaliteten. Her tester vi både korrekt login, forkert adgangskode, ukendt e-mail og manglende loginoplysninger. Vi tester også, at systemet håndterer store bogstaver og mellemrum i e-mailadressen, så brugeren ikke bliver afvist på grund af forskelle i indtastningen.

Til disse tests har vi anvendt en `FakeUserMapper`, som simulerer databaseadgangen. På den måde kan vi teste loginlogikken uden at være afhængige af en aktiv PostgreSQL-database.

Vores arbejde med MVC og Single Responsibility Principle (SRP) har gjort det lettere at teste de enkelte dele af systemet. Ved at adskille forretningslogikken fra databaseadgangen kan vi undersøge funktionaliteten mere isoleret og lettere finde frem til, hvor eventuelle fejl opstår.

7.2 Black-box-test

Ud over vores unit tests har vi udført black-box-test, hvor vi har undersøgt systemets funktionalitet gennem brugergrænsefladen uden at tage udgangspunkt i den bagvedliggende kode.

Vi har især fokuseret på almindeligt login og administratorlogin. Her har vi afprøvet forskellige kombinationer af korrekte og forkerte loginoplysninger for at kontrollere, hvordan systemet reagerer.

Vi har også undersøgt systemets fejlmeddelelser ved ugyldigt brugerinput. Under vores manuelle tests oplevede vi, at fejlmeddelelserne blev vist korrekt, når brugeren indtastede ugyldige oplysninger.

7.3 Brugertest

For at undersøge om hjemmesiden var intuitiv at anvende, fik vi nogle af vores forældre til at afprøve systemet. Formålet var at få feedback fra personer, som ikke havde været involveret i udviklingen, og derfor ikke på forhånd kendte sidernes opbygning.

Under brugertestene blev vi opmærksomme på, at enkelte elementer og knapper ikke var placeret så naturligt, som vi selv havde forventet. Det førte til ændringer i vores frontend-design, hvor vi blandt andet justerede placeringen af elementer for at gøre navigationen mere overskuelig.

Brugertestene viste os, at et design godt kan virke logisk for udviklerne, uden nødvendigvis at være lige så intuitivt for andre brugere. Derfor har feedbacken været en vigtig del af vores designproces.

7.4 Vurdering af test

Samlet set har de forskellige testformer hjulpet os med at undersøge både systemets funktionalitet og brugeroplevelse.

Unit tests har givet os mulighed for at kontrollere centrale dele af forretningslogikken, mens black-box-test har været med til at undersøge systemets funktionalitet gennem brugergrænsefladen. Brugertestene har samtidig givet os værdifuld feedback, som vi har brugt til at forbedre hjemmesidens design.

Vi har dermed arbejdet med test på flere niveauer frem for udelukkende at kontrollere, om hjemmesiden fungerer som den brude.

8. Status på implementering

Vi har fået færdigjort alle de otte user stories, og vi har vurderer, at hjemmesidens funktioner virker som de burde. Kunder kan oprette en bruger, logge ind, sammensætte cupcakes, tilføje dem til indkøbskurven og gennemføre en betaling. Kunder kan også se deres tidligere ordrer, mens administratorer har mulighed for at se og administrere kunder og ordrer.

Vi startede tidligt på projektet, hvilket gav os god tid til at arbejde med de forskellige funktioner og løbende rette fejl. Derfor har vi ikke været nødt til at fravælge nogen af vores user stories.

Undervejs har vi også arbejdet med at forbedre hjemmesidens design og brugervenlighed, blandt andet ved at justere placeringen af knapper og gøre navigationen mere overskuelig. Dette blev gjort efter nogle bruger tests, som beskrevet tidligere.

Derfor er vi samlet set tilfredse med at have noget alle funktionelle mål som opgaven har dikteret.

9. Proces

9.1 Samarbejde og planlægning

I løbet af projektet har vi arbejdet ud fra et princip med fælles programmering med individuelt arbejde. Når vi arbejdede sammen, brugte vi Code With Me, så vi kunne skrive kode i det samme projekt og diskutere løsninger undervejs. Det fungerede godt, fordi vi hurtigt kunne komme med nye idéer, ændre løsninger og opdage fejl, mens der blev programmeret. Når flere fulgte med i koden, var det nemmere at opdage problemer, som den person, der skrev koden, måske havde overset.

Arbejdsmetode gjorde, at vi kunne færdiggøre mange af vores user stories hurtigt og effektivt. Vi behøvede ikke hele tiden at oprette branches og pull requests eller vente på, at et andet gruppemedlem

gennemgik koden, før vi kunne fortsætte. En ulempe ved Code With Me var dog, at vi nogle gange måtte vente på, at en igangværende opgave blev færdig, før vi kunne begynde på den næste user story.

Når vi ikke havde mulighed for at mødes fysisk eller digitalt, fordelte vi opgaverne mellem os og arbejdede individuelt. Her brugte vi Git-branches og pull requests, så vi kunne udvikle forskellige dele af projektet uden at arbejde direkte på den samme branch. Det var især en fordel ved opgaver som CSS, hvor vi oplevede, at det fungerede bedst, hvis én person havde ansvaret ad gangen.

Vi holdt daglige møder i hverdagene kl. 16, hvor vi gennemgik, hvad vi havde lavet, og aftalte de næste opgaver. Vi brugte også GitHub Projects som Kanban-board til at holde styr på opgaverne og følge deres fremdrift.

Vi valgte som udgangspunkt at arbejde med én user story ad gangen og først gå videre, når acceptkriterierne var opfyldt. Det gjorde det lettere at holde fokus på den enkelte funktion og undgå at have for mange ufærdige opgaver på samme tid.

9.2 Roller og ansvar

Vi fordelte rollerne i gruppen for at skabe en tydelig ansvarsfordeling. Philip havde rollen som Scrum Master og stod primært for koordinering og arbejdsprocessen. Han havde desuden hovedansvaret for rapportskrivningen, selvom alle har bidraget til skrivningen.

Lucas var Product Owner og havde i starten også rollen som Tech Lead. Han arbejdede blandt andet med ER-diagrammet, forretnings logik, og black-box-test. Senere overtog Sofus en del af Tech Lead-ansvaret og arbejdede desuden med GitHub Projects og formulering af acceptkriterier, samt CSS og html.

Rollerne hjalp os med at holde styr på de forskellige ansvarsområder. Product Owner havde fokus på kravene og kundens behov, mens Tech Lead havde fokus på tekniske beslutninger og kodekvalitet. Scrum Master havde ansvaret for at understøtte samarbejdet og sørge for, at vi hele tiden havde fremdrift i projektet.

Selvom vi havde fordelt rollerne, arbejdede vi stadig sammen på tværs af ansvarsområderne. Vi diskuterede løsninger i fællesskab og hjalp hinanden, når der opstod udfordringer.

Selvom vi hver især havde vores egne ansvarsområder, var det vigtigt for os, at alle bidrog til selve programmeringen. Derfor har vi alle været med til at skrive kode og udvikle forskellige klasser og funktioner i projektet.

Det var et bevidst valg fra vores side, da vi gerne ville sikre, at alle fik praktisk erfaring med koden og kunne forklare de løsninger, vi havde lavet. Samtidig gjorde det, at vi alle kunne følge med i projektets udvikling og fik en bedre forståelse af, hvordan de forskellige dele af systemet hænger sammen.

9.3 Git-strategi, kvalitet og refleksion

Vores Git-strategi afhang af, hvordan vi arbejdede. Når vi programmerede sammen, brugte vi primært Code With Me, mens vi anvendte branches og pull requests til individuelt arbejde. Det gav os fleksibilitet til at vælge den metode, der passede bedst til den enkelte opgave.

Vi arbejdede ikke med en detaljeret Definition of Done (DoD), men brugte vores acceptkriterier som det vigtigste mål for, hvornår en user story var færdig. Når vi vurderede, at funktionaliteten virkede, og acceptkriterierne var opfyldt, gik vi videre til den næste opgave. Vi fulgte derfor nogle af principperne fra Scrum uden at anvende alle de formelle Scrum-processer.

Generelt fungerede samarbejdet godt, og vi oplevede ikke større konflikter eller problemer, som bremsede udviklingen. Den største udfordring var frontend-designet, hvor vi flere gange måtte justere placeringen af knapper, navigationen og sidernes opbygning for at gøre hjemmesiden mere brugervenlig.

Hvis vi skulle lave et lignende projekt igen, ville vi fortsat bruge Code With Me og daglige statusmøder, da begge dele fungerede godt for os. Vi ville dog være mere systematiske med at dokumentere vores testcases og designbeslutninger. Det ville gøre det nemmere at holde styr på ændringerne og efterfølgende forklare, hvorfor vi valgte bestemte løsninger, og hvordan vi havde testet dem.

10. Konklusion

I dette projekt har vi udviklet en webshop til Copenhagen Cupcakes, hvor kunder kan sammensætte og bestille deres egne cupcakes. Vi har fået implementeret alle otte user stories og vurderer derfor, at vi har opfyldt de funktionelle krav, vi satte os i starten af projektet. Hjemmesiden giver kunder mulighed for at oprette en konto, logge ind, vælge cupcakes, bruge indkøbskurven og gennemføre en betaling. Derudover har vi lavet en administrationsdel, hvor administratorer kan se og administrere kunder og ordrer.

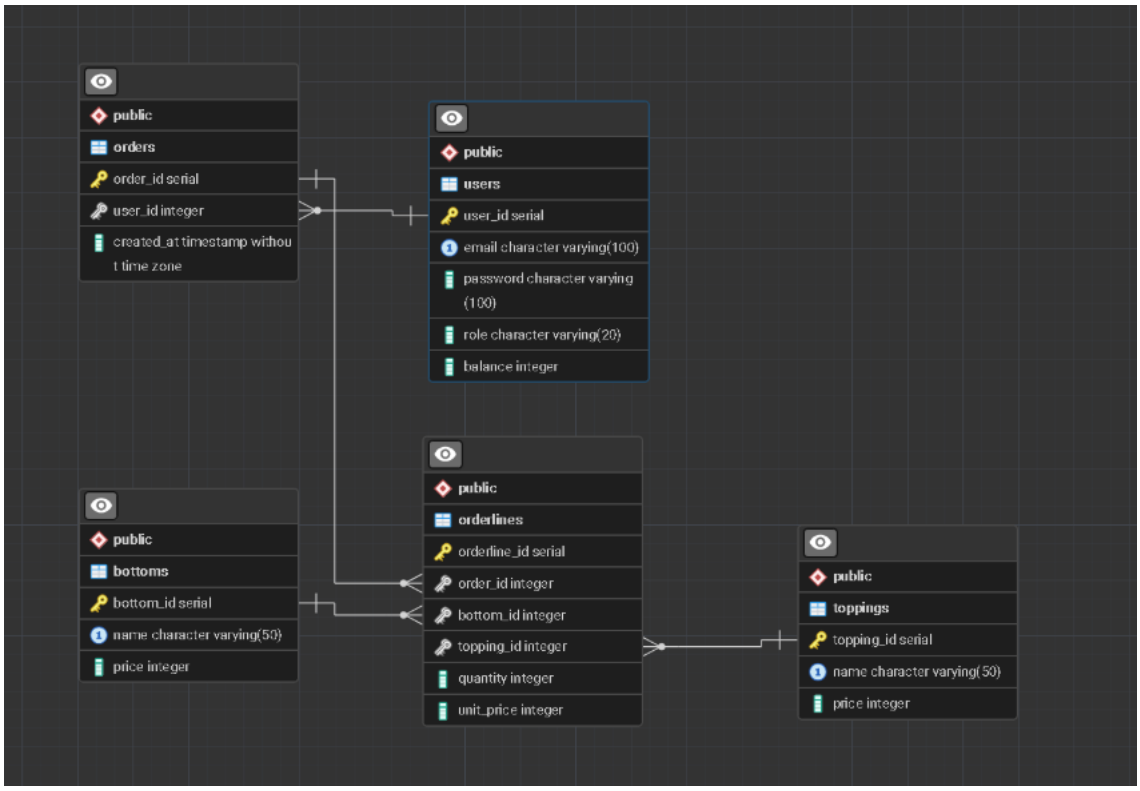
Under udviklingen har vi arbejdet med Java, Javalin, Thymeleaf og PostgreSQL. Vi har især haft fokus på MVC og SRP for at skabe en overskuelig kodebase, hvor controllers, services og mappers har forskellige ansvarsområder. Vi har også arbejdet med validering, fejlhåndtering og sikkerhed, blandt andet gennem BCrypt, PreparedStatement og databasetransaktioner.

Vi har samtidig lært meget om samarbejde og projektstyring. Ved at kombinere Code With Me med Git-branches og pull requests har vi kunnet arbejde både sammen og individuelt. Vi har desuden brugt GitHub Projects til at holde styr på vores user stories og deres fremdrift. Selvom vi havde fordelt rollerne mellem os, har alle bidraget til programmeringen, så vi hver især har fået en forståelse af projektets kode og opbygning.

En af de største udfordringer har været at gøre hjemmesiden brugervenlig. Vi har derfor løbende justeret designet og navigationen ud fra vores egne vurderinger og feedback fra testpersoner. Det har vist os, at en løsning ikke kun skal fungere teknisk, men også være nem for brugeren at forstå.

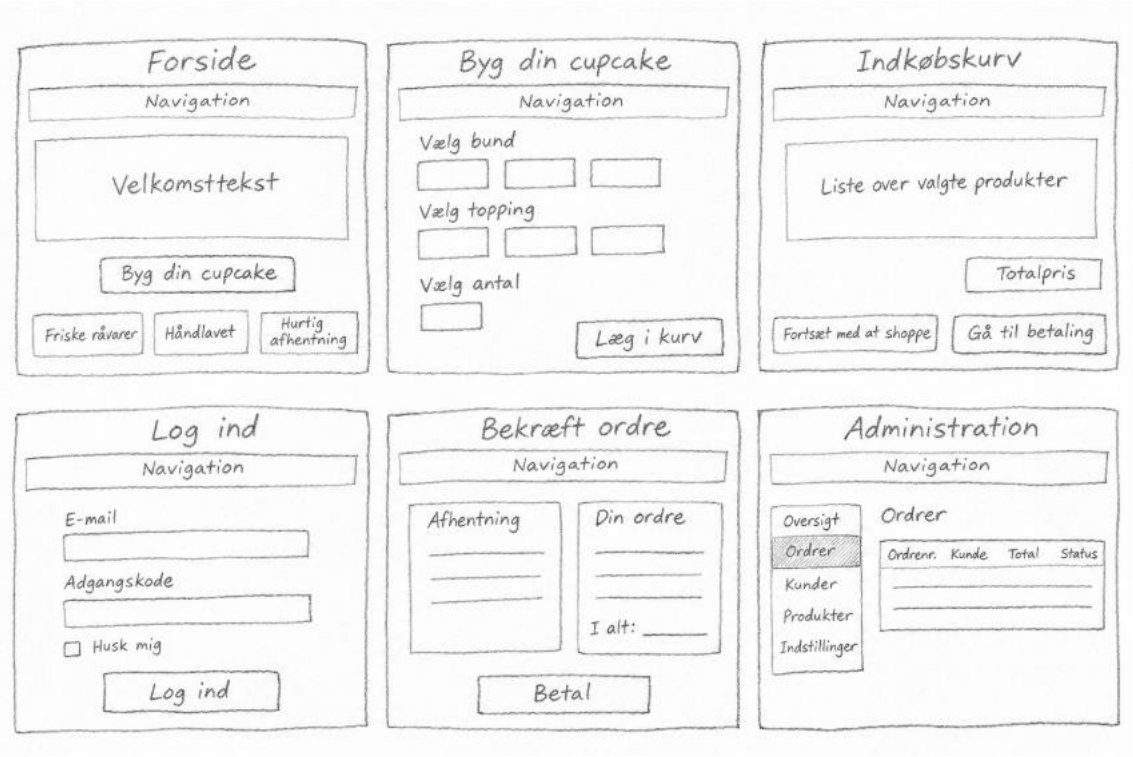
Samlet set er vi tilfredse med resultatet og den måde, vi har arbejdet på. Projektet har givet os praktisk erfaring med at udvikle en webapplikation fra database til brugergrænseflade, samtidig med at vi har fået en bedre forståelse af, hvordan tekniske valg, test og samarbejde hænger sammen i et større projekt. Hvis vi skulle arbejde videre med løsningen, ville vi især fokusere på mere systematisk test og yderligere forbedringer af brugeroplevelsen.

Bilag A – ER-diagram



Bilag A

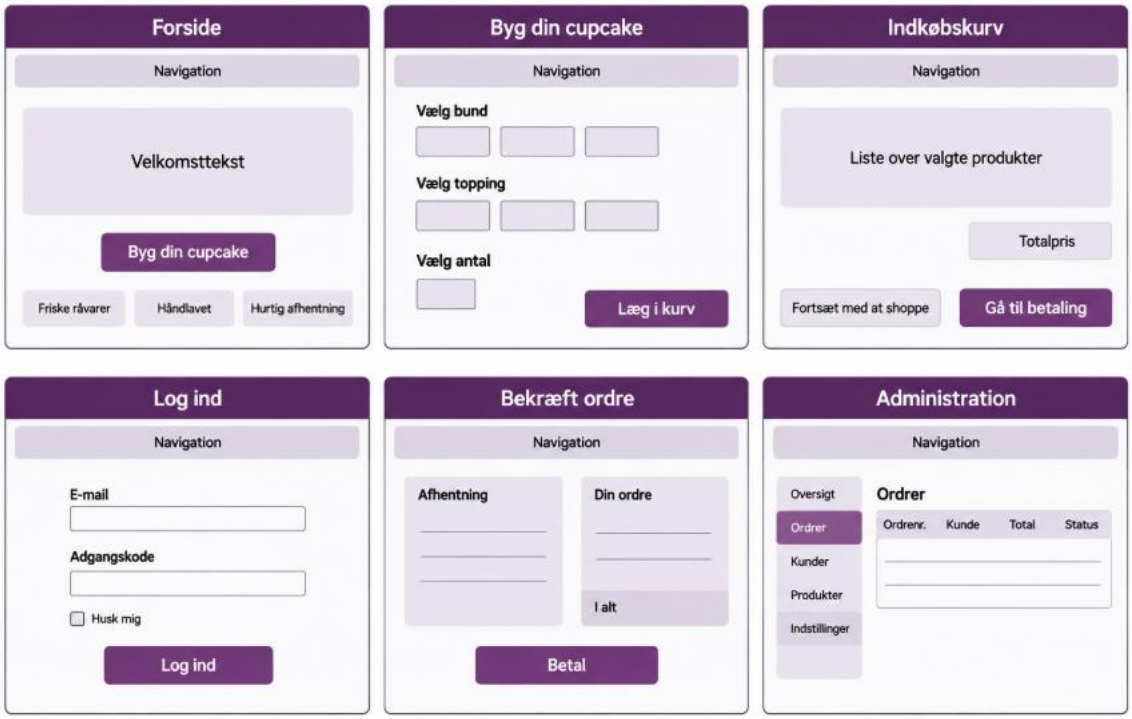
Bilag B – Mockup og designiteration



Bilag B

Note: Billedet er tegnet i og skrevet i hånden og taget billede af og lagt op, derefter har vi fået en AI til at, gøre papiret pænere samt fjerne krøller samt skygger osv.

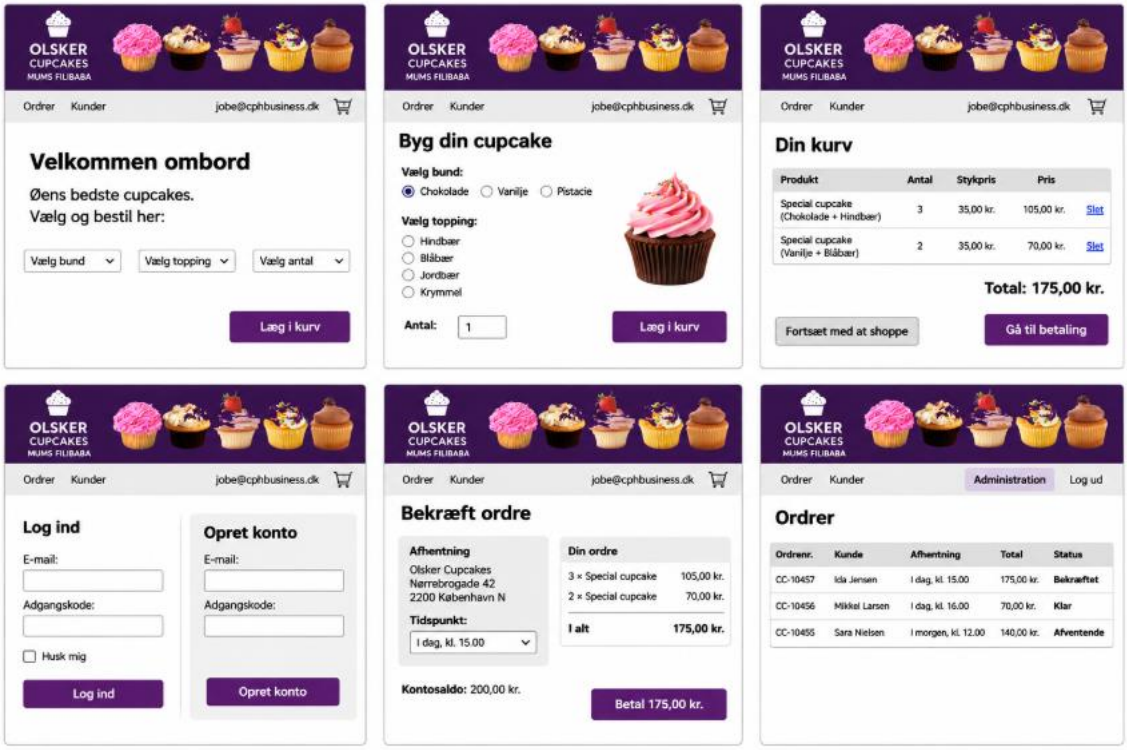
Bilag C – Mockup og designiteration



Bilag C.

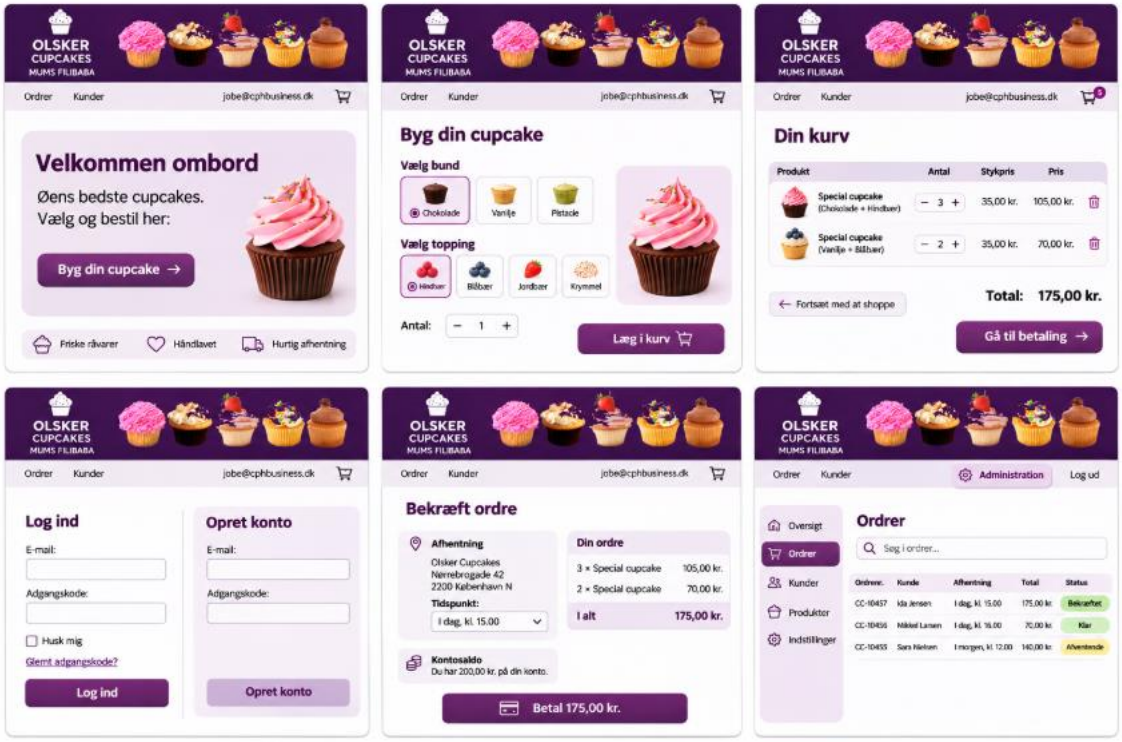
En Ai har brugt bilag b' til at skabe en digital version af mockup'en. Dette blev gjort for at skabe et mere klar overblik og digitalt over blik, for at kunne tage det videre til Figma.

Bilag D – Mockup og designiteration



Bilag D.

Bilag E – Mockup og designiteration



Bilag E. Indsendt billede fra gruppens designproces.

Bilag F – Teknisk dokumentation og afleveringskontrol

Kodegrundlag: Cupcake-project-2026.zip. Centrale filer: pom.xml, sql/cuocakeinit.sql, controllers/UserController.java, controllers/ShopController.java, controllers/AdminController.java, persistence/OrderMapper.java, services/UserService.java og controllers/ViewModel.java.

Inden aflevering: Tilføj klasse, fulde navne og studie-e-mails; tilføj GitHub-repository og videolink; indsæt egentligt UML-domæne- og navigationsdiagram; sammenhold acceptkriterier med GitHub Projects; opdater Word-indholdsfortegnelse og eksporter til rapport.pdf i mappen rapport.